StreamingApp

DevOps CI/CD Implementation Plan

Jenkins | Docker | Kubernetes | AWS EKS | Terraform | Ansible

Prometheus | Grafana | Helm | AWS ECR | AWS S3

Project Information

Project:	Capstone - DevOps CI/CD Pipeline
Application:	StreamingApp (Microservices - React + Node.js)
Cloud:	AWS (EKS, ECR, S3, VPC, EC2)
Prepared by:	Apoorva Deshpande
Date:	June 2026
Version:	1.0

Evaluation Criteria

Documentation	**15%**
Implementation	**75%**
Cost Optimization	**10%**

1. Application Architecture Overview

1.1 Microservices Architecture

StreamingApp is built as a microservices application consisting of five independent services orchestrated via Docker Compose locally and Kubernetes (AWS EKS) in production. Each service is independently deployable, scalable, and communicates via REST APIs and WebSockets.

Service	Port | Technology | Responsibility
Frontend	80 (Nginx) | React 18 + MUI | SPA served via Nginx reverse-proxy
Auth Service	3001 | Node.js/Express + Mongoose | User registration, login, JWT, password reset
Streaming Service	3002 | Node.js/Express + AWS S3 | Video catalogue, presigned URLs, playback
Admin Service	3003 | Node.js/Express + Multer + S3 | Video upload/management by admin roles
Chat Service	3004 | Node.js/Express + Socket.IO | Real-time WebSocket chat during streams
MongoDB	27017 | MongoDB 6 | Shared database (one DB per service namespace)

1.2 AWS Production Architecture

The production environment runs on AWS using EKS for container orchestration, ECR for image storage, S3 for video assets, and an Application Load Balancer for ingress routing.

* AWS EKS (Elastic Kubernetes Service) - hosts all 5 application pods
* AWS ECR (Elastic Container Registry) - stores Docker images for each service
* AWS S3 - stores video content; served via CloudFront CDN (optional)
* AWS ALB (Application Load Balancer) - routes external traffic to Kubernetes Ingress
* AWS VPC - isolated network with public/private subnets across 2 AZs
* AWS DocumentDB (optional) or self-managed MongoDB on EKS via StatefulSet
* AWS Secrets Manager - stores JWT_SECRET, DB credentials, AWS keys
* Prometheus + Grafana - deployed in monitoring namespace on EKS
* Jenkins - runs on dedicated EC2 instance (t3.medium) for CI/CD orchestration

1.3 Network Flow

User -> ALB -> Nginx Ingress Controller -> Frontend Pod (port 80)
Frontend -> ALB -> Nginx Ingress -> Auth/Streaming/Admin/Chat Services
Chat Service <- WebSocket -> Browser via Ingress (ws:// upgrade)
Services -> MongoDB (ClusterIP Service, port 27017)
Services -> AWS S3 (via IAM Role / Pod Identity)

2. Dockerfiles - All Services

2.1 Frontend Dockerfile (React -> Nginx)

Multi-stage build: Stage 1 builds the React app with Node 18; Stage 2 copies the static output into a minimal Nginx image. Environment variables are injected at build time so the compiled JS bundle knows each backend service URL.

frontend/Dockerfile

```
# -- Stage 1: Build -----------------------------------------------
FROM node:18-alpine AS builder
WORKDIR /app

ARG REACT_APP_AUTH_SERVICE_URL
ARG REACT_APP_STREAMING_SERVICE_URL
ARG REACT_APP_ADMIN_SERVICE_URL
ARG REACT_APP_CHAT_SERVICE_URL

ENV REACT_APP_AUTH_SERVICE_URL=$REACT_APP_AUTH_SERVICE_URL
ENV REACT_APP_STREAMING_SERVICE_URL=$REACT_APP_STREAMING_SERVICE_URL
ENV REACT_APP_ADMIN_SERVICE_URL=$REACT_APP_ADMIN_SERVICE_URL
ENV REACT_APP_CHAT_SERVICE_URL=$REACT_APP_CHAT_SERVICE_URL

COPY package*.json ./
RUN npm ci --only=production=false
COPY . .
RUN npm run build

# -- Stage 2: Serve -----------------------------------------------
FROM nginx:1.27-alpine
COPY --from=builder /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost/health || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

Nginx Config (frontend/nginx.conf) - required for React Router SPA routing

frontend/nginx.conf

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    # Health-check endpoint for K8s liveness probe
    location /health { return 200 'ok'; add_header Content-Type text/plain; }

    # Proxy API calls to backend services (K8s Service DNS names)
    location /api/auth/        { proxy_pass http://auth-service:3001/;   }
    location /api/streaming/   { proxy_pass http://streaming-service:3002/; }
    location /api/admin/       { proxy_pass http://admin-service:3003/;  }
    location /api/chat/        { proxy_pass http://chat-service:3004/;   }
```

```
    # WebSocket upgrade for Socket.IO
    location /socket.io/ {
        proxy_pass         http://chat-service:3004/socket.io/;
        proxy_http_version 1.1;
        proxy_set_header   Upgrade $http_upgrade;
        proxy_set_header   Connection "upgrade";
    }

    # SPA fallback
    location / { try_files $uri $uri/ /index.html; }
}
```

2.2 Auth Service Dockerfile

backend/authService/Dockerfile

```
FROM node:18-alpine
WORKDIR /app

# Install deps first (layer cache)
COPY package*.json ./
RUN npm ci --only=production

# Copy source
COPY . .

# Non-root user for security
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 3001
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s \
  CMD wget -qO- http://localhost:3001/api/health || exit 1

CMD ["node", "index.js"]
```

2.3 Streaming Service Dockerfile

backend/streamingService/Dockerfile

```
FROM node:18-alpine
WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production

COPY . .

RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 3002
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s \
  CMD wget -qO- http://localhost:3002/api/health || exit 1

CMD ["node", "index.js"]
```

2.4 Admin Service Dockerfile

backend/adminService/Dockerfile

```
FROM node:18-alpine
WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production
```

```
COPY . .

RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 3003
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s \
  CMD wget -qO- http://localhost:3003/api/health || exit 1

CMD ["node", "index.js"]
```

2.5 Chat Service Dockerfile

backend/chatService/Dockerfile

```
FROM node:18-alpine
WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production

COPY . .

RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 3004
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s \
  CMD wget -qO- http://localhost:3004/api/health || exit 1

CMD ["node", "index.js"]
```

3. Kubernetes YAML Manifests

3.1 Namespace

k8s/namespace.yaml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: streamingapp
  labels:
    app.kubernetes.io/managed-by: helm
```

3.2 Secrets

Never commit real secrets. Create the secret with kubectl or AWS Secrets Manager CSI driver. The YAML below shows the structure; base64-encoded values are placeholders.

k8s/secrets.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: streamingapp-secrets
  namespace: streamingapp
type: Opaque
stringData:          # use stringData so values are NOT manually base64-encoded
  JWT_SECRET: "REPLACE_ME"
  MONGO_URI: "mongodb://mongo-service:27017/streamingapp"
  AWS_ACCESS_KEY_ID: "REPLACE_ME"
  AWS_SECRET_ACCESS_KEY: "REPLACE_ME"
  AWS_REGION: "us-east-1"
  AWS_S3_BUCKET: "streamingapp-videos"
  AWS_CDN_URL: "https://REPLACE.cloudfront.net"
  CLIENT_URLS: "https://streamingapp.example.com"
  STREAMING_PUBLIC_URL: "https://streamingapp.example.com"
```

3.3 MongoDB StatefulSet (Self-managed)

k8s/mongo-statefulset.yaml

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
  namespace: streamingapp
spec:
  serviceName: mongo-service
  replicas: 1
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
```

```
      labels:
        app: mongo
    spec:
      containers:
        - name: mongo
          image: mongo:6
          ports:
            - containerPort: 27017
          volumeMounts:
            - name: mongo-data
              mountPath: /data/db
          resources:
            requests:
              cpu: 250m
              memory: 512Mi
            limits:
              cpu: 500m
              memory: 1Gi
  volumeClaimTemplates:
    - metadata:
        name: mongo-data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 20Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
  namespace: streamingapp
spec:
  clusterIP: None      # headless for StatefulSet DNS
  selector:
    app: mongo
  ports:
    - port: 27017
      targetPort: 27017
```

3.4 Auth Service Deployment & Service

k8s/auth-service.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: streamingapp
  labels:
    app: auth-service
    version: "1.0"
spec:
  replicas: 2
  selector:
    matchLabels:
      app: auth-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: auth-service
    spec:
      containers:
        - name: auth-service
          image: <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/streamingapp/auth-service:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 3001
          envFrom:
            - secretRef:
                name: streamingapp-secrets
          env:
            - name: PORT
              value: "3001"
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 300m
              memory: 256Mi
          livenessProbe:
            httpGet:
              path: /api/health
              port: 3001
            initialDelaySeconds: 20
            periodSeconds: 15
          readinessProbe:
            httpGet:
```

```
              path: /api/health
              port: 3001
            initialDelaySeconds: 10
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: auth-service
  namespace: streamingapp
spec:
  selector:
    app: auth-service
  ports:
    - port: 3001
      targetPort: 3001
  type: ClusterIP
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: auth-service-hpa
  namespace: streamingapp
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: auth-service
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

3.5 Streaming Service Deployment & Service

k8s/streaming-service.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: streaming-service
  namespace: streamingapp
  labels:
    app: streaming-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: streaming-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: streaming-service
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port:   "3002"
        prometheus.io/path:   "/metrics"
    spec:
      containers:
        - name: streaming-service
          image: <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/streamingapp/streaming-service:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 3002
          envFrom:
            - secretRef:
                name: streamingapp-secrets
          env:
            - name: PORT
              value: "3002"
          resources:
            requests:
              cpu: 200m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
          livenessProbe:
            httpGet:
              path: /api/health
              port: 3002
            initialDelaySeconds: 20
```

```
            periodSeconds: 15
          readinessProbe:
            httpGet:
              path: /api/health
              port: 3002
            initialDelaySeconds: 10
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: streaming-service
  namespace: streamingapp
spec:
  selector:
    app: streaming-service
  ports:
    - port: 3002
      targetPort: 3002
  type: ClusterIP
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: streaming-service-hpa
  namespace: streamingapp
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: streaming-service
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

3.6 Admin Service Deployment & Service

k8s/admin-service.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: admin-service
  namespace: streamingapp
  labels:
    app: admin-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: admin-service
  template:
    metadata:
      labels:
        app: admin-service
    spec:
      containers:
        - name: admin-service
          image: <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/streamingapp/admin-service:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 3003
          envFrom:
            - secretRef:
                name: streamingapp-secrets
          env:
            - name: PORT
              value: "3003"
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 300m
              memory: 256Mi
          livenessProbe:
            httpGet:
              path: /api/health
              port: 3003
            initialDelaySeconds: 20
            periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
  name: admin-service
  namespace: streamingapp
spec:
  selector:
```

```
    app: admin-service
  ports:
    - port: 3003
      targetPort: 3003
  type: ClusterIP
```

3.7 Chat Service Deployment & Service

k8s/chat-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: chat-service
  namespace: streamingapp
  labels:
    app: chat-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: chat-service
  template:
    metadata:
      labels:
        app: chat-service
    spec:
      containers:
        - name: chat-service
          image: <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/streamingapp/chat-service:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 3004
          envFrom:
            - secretRef:
                name: streamingapp-secrets
          env:
            - name: PORT
              value: "3004"
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 300m
              memory: 256Mi
          livenessProbe:
            tcpSocket:
              port: 3004
            initialDelaySeconds: 20
            periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
```

```
  name: chat-service
  namespace: streamingapp
spec:
  selector:
    app: chat-service
  ports:
    - port: 3004
      targetPort: 3004
  type: ClusterIP
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: chat-service-hpa
  namespace: streamingapp
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: chat-service
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

3.8 Frontend Deployment & Service

k8s/frontend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: streamingapp
  labels:
    app: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/streamingapp/frontend:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 128Mi
          livenessProbe:
            httpGet:
              path: /health
              port: 80
            initialDelaySeconds: 10
            periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: streamingapp
spec:
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

3.9 Ingress (AWS ALB Ingress Controller)

k8s/ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: streamingapp-ingress
  namespace: streamingapp
  annotations:
    kubernetes.io/ingress.class: "alb"
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-east-1:ACCOUNT:certificate/CERT-ID
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'
    alb.ingress.kubernetes.io/ssl-redirect: "443"
    alb.ingress.kubernetes.io/healthcheck-path: /health
spec:
  rules:
    - host: streamingapp.example.com
      http:
        paths:
          - path: /api/auth
            pathType: Prefix
            backend:
              service:
                name: auth-service
                port:
                  number: 3001
          - path: /api/streaming
            pathType: Prefix
            backend:
              service:
                name: streaming-service
                port:
                  number: 3002
          - path: /api/admin
            pathType: Prefix
            backend:
              service:
                name: admin-service
                port:
                  number: 3003
          - path: /socket.io
            pathType: Prefix
            backend:
              service:
                name: chat-service
                port:
                  number: 3004
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
```

```
    port:
      number: 80
```

4. Helm Chart Structure

A single Helm chart (streamingapp) parameterizes all five services. Each service gets its own named section in values.yaml so the same chart is reused across dev/staging/prod by swapping a values override file.

4.1 Chart.yaml

helm/streamingapp/Chart.yaml

```
apiVersion: v2
name: streamingapp
description: StreamingApp microservices chart - React SPA + 4 Node.js services
type: application
version: 1.0.0
appVersion: "1.0"
keywords:
  - streaming
  - nodejs
  - react
maintainers:
  - name: Apoorva Deshpande
```

4.2 values.yaml (abridged - key sections)

helm/streamingapp/values.yaml

```
global:
  imageRegistry: <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com
  imageTag: latest
  namespace: streamingapp
  secretName: streamingapp-secrets

frontend:
  enabled: true
  replicaCount: 2
  image:
    name: streamingapp/frontend
  service:
    port: 80
  resources:
    requests:  { cpu: 50m,  memory: 64Mi  }
    limits:    { cpu: 200m, memory: 128Mi }

authService:
  enabled: true
  replicaCount: 2
  image:
    name: streamingapp/auth-service
  service:
    port: 3001
  hpa:
    minReplicas: 2
    maxReplicas: 6
    cpuUtilization: 70
```

```
  resources:
    requests:  { cpu: 100m, memory: 128Mi }
    limits:    { cpu: 300m, memory: 256Mi }

streamingService:
  enabled: true
  replicaCount: 2
  image:
    name: streamingapp/streaming-service
  service:
    port: 3002
  hpa:
    minReplicas: 2
    maxReplicas: 8
    cpuUtilization: 60
  resources:
    requests:  { cpu: 200m, memory: 256Mi }
    limits:    { cpu: 500m, memory: 512Mi }

adminService:
  enabled: true
  replicaCount: 1
  image:
    name: streamingapp/admin-service
  service:
    port: 3003
  resources:
    requests:  { cpu: 100m, memory: 128Mi }
    limits:    { cpu: 300m, memory: 256Mi }

chatService:
  enabled: true
  replicaCount: 2
  image:
    name: streamingapp/chat-service
  service:
    port: 3004
  hpa:
    minReplicas: 2
    maxReplicas: 6
    cpuUtilization: 70

mongodb:
  enabled: true
  storageSize: 20Gi

ingress:
  enabled: true
  host: streamingapp.example.com
  certArn: arn:aws:acm:us-east-1:ACCOUNT:certificate/CERT-ID
```

4.3 templates/deployment.yaml (generic template)

helm/streamingapp/templates/deployment.yaml

```
{{- range $svcName, $svc := .Values.services }}
{{- if $svc.enabled }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ $svcName }}
  namespace: {{ $.Values.global.namespace }}
  labels:
    app: {{ $svcName }}
    chart: {{ $.Chart.Name }}-{{ $.Chart.Version }}
spec:
  replicas: {{ $svc.replicaCount }}
  selector:
    matchLabels:
      app: {{ $svcName }}
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: {{ $svcName }}
    spec:
      containers:
        - name: {{ $svcName }}
          image: "{{ $.Values.global.imageRegistry }}/{{ $svc.image.name }}:{{ $.Values.global.imageTag }}"
          imagePullPolicy: Always
          ports:
            - containerPort: {{ $svc.service.port }}
          envFrom:
            - secretRef:
                name: {{ $.Values.global.secretName }}
          resources: {{ toYaml $svc.resources | nindent 12 }}
          livenessProbe:
            httpGet:
              path: /api/health
              port: {{ $svc.service.port }}
            initialDelaySeconds: 20
            periodSeconds: 15
          readinessProbe:
            httpGet:
              path: /api/health
              port: {{ $svc.service.port }}
            initialDelaySeconds: 10
            periodSeconds: 10
---
{{- end }}
{{- end }}
```

4.4 templates/hpa.yaml

helm/streamingapp/templates/hpa.yaml

```
{{- range $svcName, $svc := .Values.services }}
{{- if and $svc.enabled $svc.hpa }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ $svcName }}-hpa
  namespace: {{ $.Values.global.namespace }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ $svcName }}
  minReplicas: {{ $svc.hpa.minReplicas }}
  maxReplicas: {{ $svc.hpa.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ $svc.hpa.cpuUtilization }}
---
{{- end }}
{{- end }}
```

4.5 values-prod.yaml (Production Overrides)

helm/values-prod.yaml

```
global:
  imageTag: "REPLACED_BY_JENKINS_AT_BUILD_TIME"

authService:
  replicaCount: 3

streamingService:
  replicaCount: 4
  hpa:
    maxReplicas: 12

chatService:
  replicaCount: 3
```

5. Jenkins CI/CD Pipeline (Jenkinsfile)

The Jenkinsfile defines a declarative multi-stage pipeline. It is stored at the repository root so Jenkins picks it up automatically when a webhook fires on any push to main or a pull request.

5.1 Pipeline Overview

Stage 1 - Checkout	Clone repository from GitHub/SCM
Stage 2 - Unit Tests	Run npm test for each Node.js service
Stage 3 - Docker Build & Push	Build images for all 5 services, push to AWS ECR
Stage 4 - Terraform Plan	Run terraform plan to preview infrastructure changes
Stage 5 - Terraform Apply	Apply infrastructure changes (EKS, VPC, Security Groups)
Stage 6 - Ansible Configure	Run Ansible playbooks to configure EC2/nodes
Stage 7 - Deploy to EKS	helm upgrade --install to deploy/update pods on EKS
Stage 8 - Smoke Test	Run curl health checks against all live services
Stage 9 - Notifications	Send Slack/email on success or failure

5.2 Jenkinsfile

Jenkinsfile

```
pipeline {
  agent any

  environment {
    AWS_ACCOUNT_ID     = credentials('aws-account-id')
    AWS_DEFAULT_REGION = 'us-east-1'
    ECR_REGISTRY       = "${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_DEFAULT_REGION}.amazonaws.com"
    IMAGE_TAG          = "${GIT_COMMIT[0..7]}"
    EKS_CLUSTER        = 'streamingapp-cluster'
    NAMESPACE          = 'streamingapp'
    SLACK_CHANNEL      = '#devops-alerts'
  }

  options {
    buildDiscarder(logRotator(numToKeepStr: '10'))
    timeout(time: 45, unit: 'MINUTES')
    disableConcurrentBuilds()
  }

  stages {
    // -- Stage 1: Checkout ------------------------------------------
    stage('Checkout') {
      steps {
        checkout scm
        sh 'git log --oneline -5'
      }
    }

    // -- Stage 2: Unit Tests ----------------------------------------
    stage('Unit Tests') {
      parallel {
```

```
        stage('Test Auth Service') {
          steps {
            dir('backend/authService') {
              sh 'npm ci && npm test -- --watchAll=false'
            }
          }
        }
        stage('Test Streaming Service') {
          steps {
            dir('backend/streamingService') {
              sh 'npm ci && npm test -- --watchAll=false'
            }
          }
        }
        stage('Test Frontend') {
          steps {
            dir('frontend') {
              sh 'npm ci && npm test -- --watchAll=false --passWithNoTests'
            }
          }
        }
      }
    }

    // -- Stage 3: Docker Build & Push ------------------------------
    stage('Docker Build & Push') {
      steps {
        script {
          sh """
            aws ecr get-login-password --region ${AWS_DEFAULT_REGION} \
              | docker login --username AWS --password-stdin ${ECR_REGISTRY}
          """
          def services = [
            [name: 'frontend',          context: 'frontend'],
            [name: 'auth-service',      context: 'backend/authService'],
            [name: 'streaming-service', context: 'backend/streamingService'],
            [name: 'admin-service',     context: 'backend/adminService'],
            [name: 'chat-service',      context: 'backend/chatService'],
          ]
          services.each { svc ->
            def img = "${ECR_REGISTRY}/streamingapp/${svc.name}:${IMAGE_TAG}"
            sh "docker build -t ${img} ${svc.context}"
            sh "docker push ${img}"
            sh "docker tag ${img} ${ECR_REGISTRY}/streamingapp/${svc.name}:latest"
            sh "docker push ${ECR_REGISTRY}/streamingapp/${svc.name}:latest"
          }
        }
      }
    }

    // -- Stage 4: Terraform Plan -----------------------------------
    stage('Terraform Plan') {
      when { branch 'main' }
      steps {
```

```
        dir('terraform') {
          withCredentials([[$class:'AmazonWebServicesCredentialsBinding',
                            credentialsId:'aws-credentials']]) {
            sh '''
              terraform init -backend-config="bucket=streamingapp-tfstate" \
                             -backend-config="key=eks/terraform.tfstate" \
                             -backend-config="region=us-east-1"
              terraform plan -out=tfplan
            '''
          }
        }
      }
    }

    // -- Stage 5: Terraform Apply ----------------------------------
    stage('Terraform Apply') {
      when { branch 'main' }
      input { message "Apply Terraform changes?" }
      steps {
        dir('terraform') {
          withCredentials([[$class:'AmazonWebServicesCredentialsBinding',
                            credentialsId:'aws-credentials']]) {
            sh 'terraform apply -auto-approve tfplan'
          }
        }
      }
    }

    // -- Stage 6: Ansible Configuration ----------------------------
    stage('Ansible Configure') {
      when { branch 'main' }
      steps {
        dir('ansible') {
          sh 'ansible-playbook -i inventory/aws_ec2.yml site.yml --extra-vars "env=prod"'
        }
      }
    }

    // -- Stage 7: Deploy to EKS ------------------------------------
    stage('Deploy to EKS') {
      steps {
        sh """
          aws eks update-kubeconfig --name ${EKS_CLUSTER} --region ${AWS_DEFAULT_REGION}
          helm upgrade --install streamingapp ./helm/streamingapp \
            --namespace ${NAMESPACE} --create-namespace \
            -f helm/values-prod.yaml \
            --set global.imageTag=${IMAGE_TAG} \
            --wait --timeout 5m
        """
      }
    }

    // -- Stage 8: Smoke Tests --------------------------------------
    stage('Smoke Tests') {
```

```
      steps {
        sh '''
          BASE=https://streamingapp.example.com
          for path in /api/auth/health /api/streaming/health; do
            STATUS=$(curl -s -o /dev/null -w "%{http_code}" ${BASE}${path})
            [ "$STATUS" = "200" ] || { echo "FAIL: ${path} returned ${STATUS}"; exit 1; }
          done
          echo "All smoke tests passed"
        '''
      }
    }
  }

  post {
    success {
      slackSend channel: env.SLACK_CHANNEL, color: 'good',
        message: ":white_check_mark: Build #${BUILD_NUMBER} on ${BRANCH_NAME} deployed successfully. Tag: ${IMAGE_TAG}"
    }
    failure {
      slackSend channel: env.SLACK_CHANNEL, color: 'danger',
        message: ":x: Build #${BUILD_NUMBER} on ${BRANCH_NAME} FAILED. Check: ${BUILD_URL}"
    }
    always {
      cleanWs()
    }
  }
}
```

6. Terraform - AWS Infrastructure

All AWS resources are defined as code. Terraform modules are stored in the /terraform directory. State is kept in an S3 bucket with DynamoDB locking for team collaboration.

6.1 Directory Structure

Directory: /terraform

```
terraform/
+-- main.tf          # Provider config, backend (S3 state)
+-- variables.tf     # Input variables
+-- outputs.tf       # EKS endpoint, VPC ID, etc.
+-- vpc.tf           # VPC, subnets, NAT gateway
+-- eks.tf           # EKS cluster + managed node group
+-- ecr.tf           # ECR repositories for each service
+-- s3.tf            # S3 bucket for video storage
+-- iam.tf           # IAM roles for EKS nodes & pods
+-- security.tf      # Security groups
```

6.2 main.tf - Provider & Backend

terraform/main.tf

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {
    bucket         = "streamingapp-tfstate"
    key            = "eks/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "streamingapp-tflock"
  }
}

provider "aws" {
  region = var.aws_region
  default_tags {
    tags = {
      Project     = "StreamingApp"
      Environment = var.environment
      ManagedBy   = "Terraform"
    }
  }
}
```

6.3 variables.tf

terraform/variables.tf

```
variable "aws_region"    { default = "us-east-1" }
variable "environment"   { default = "production" }
variable "cluster_name"  { default = "streamingapp-cluster" }
variable "cluster_version"{ default = "1.29" }
variable "vpc_cidr"      { default = "10.0.0.0/16" }
variable "private_subnets"{ default = ["10.0.1.0/24","10.0.2.0/24"] }
variable "public_subnets" { default = ["10.0.101.0/24","10.0.102.0/24"] }
variable "node_instance"  { default = "t3.medium" }
variable "node_min"       { default = 2 }
variable "node_max"       { default = 6 }
variable "node_desired"   { default = 2 }
variable "s3_bucket_name" { default = "streamingapp-videos" }
```

6.4 vpc.tf

terraform/vpc.tf

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"

  name = "streamingapp-vpc"
  cidr = var.vpc_cidr

  azs             = ["${var.aws_region}a", "${var.aws_region}b"]
  private_subnets = var.private_subnets
  public_subnets  = var.public_subnets

  enable_nat_gateway   = true
  single_nat_gateway   = true   # cost optimisation for non-HA staging
  enable_dns_hostnames = true

  # EKS ALB controller discovery tags
  public_subnet_tags  = { "kubernetes.io/role/elb" = 1 }
  private_subnet_tags = { "kubernetes.io/role/internal-elb" = 1 }
}
```

6.5 eks.tf

terraform/eks.tf

```
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "~> 20.0"

  cluster_name    = var.cluster_name
  cluster_version = var.cluster_version

  vpc_id     = module.vpc.vpc_id
  subnet_ids = module.vpc.private_subnets

  cluster_endpoint_public_access = true

  eks_managed_node_groups = {
```

```
    default = {
      name           = "streamingapp-ng"
      instance_types = [var.node_instance]
      min_size       = var.node_min
      max_size       = var.node_max
      desired_size   = var.node_desired
      disk_size      = 20
      labels         = { Environment = var.environment }
      tags           = { AutoScaling = "enabled" }
    }
  }

  cluster_addons = {
    coredns    = { most_recent = true }
    kube-proxy = { most_recent = true }
    vpc-cni    = { most_recent = true }
  }
}
```

6.6 ecr.tf - One Repo per Microservice

terraform/ecr.tf

```
locals {
  services = ["frontend","auth-service","streaming-service","admin-service","chat-service"]
}

resource "aws_ecr_repository" "services" {
  for_each             = toset(local.services)
  name                 = "streamingapp/${each.key}"
  image_tag_mutability = "MUTABLE"

  image_scanning_configuration { scan_on_push = true }

  lifecycle { prevent_destroy = true }
}

resource "aws_ecr_lifecycle_policy" "keep_last_10" {
  for_each   = aws_ecr_repository.services
  repository = each.value.name

  policy = jsonencode({
    rules = [{
      rulePriority = 1
      description  = "Keep last 10 images"
      selection    = { tagStatus = "any", countType = "imageCountMoreThan", countNumber = 10 }
      action       = { type = "expire" }
    }]
  })
}
```

7. Ansible - Configuration Management

Ansible playbooks configure the Jenkins EC2 instance and any worker nodes. The AWS EC2 dynamic inventory plugin discovers hosts automatically using tags.

7.1 Directory Structure

ansible/

```
ansible/
+-- site.yml                    # Master playbook
+-- ansible.cfg
+-- inventory/
|   +-- aws_ec2.yml             # Dynamic inventory (AWS)
+-- roles/
    +-- common/
    |   +-- tasks/main.yml      # OS updates, time-zone, users
    |   +-- handlers/main.yml
    +-- docker/
    |   +-- tasks/main.yml      # Install Docker CE
    +-- kubectl/
    |   +-- tasks/main.yml      # Install kubectl, Helm, awscli
    +-- jenkins/
        +-- tasks/main.yml      # Install Jenkins, plugins
```

7.2 site.yml - Master Playbook

ansible/site.yml

```
---
- name: Configure Jenkins CI Server
  hosts: tag_Role_jenkins
  become: true
  roles:
    - common
    - docker
    - kubectl
    - jenkins

- name: Configure Application Nodes
  hosts: tag_Role_appnode
  become: true
  roles:
    - common
    - docker
```

7.3 roles/docker/tasks/main.yml

ansible/roles/docker/tasks/main.yml

```
---
- name: Install prerequisites
  apt:
    name: [apt-transport-https, ca-certificates, curl, gnupg, lsb-release]
```

```
    state: present
    update_cache: yes

- name: Add Docker GPG key
  apt_key:
    url: https://download.docker.com/linux/ubuntu/gpg
    state: present

- name: Add Docker repository
  apt_repository:
    repo: "deb [arch=amd64] https://download.docker.com/linux/ubuntu {{ ansible_lsb.codename }} stable"

- name: Install Docker CE
  apt:
    name: [docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin]
    state: present

- name: Enable and start Docker
  systemd:
    name: docker
    enabled: true
    state: started

- name: Add Jenkins user to docker group
  user:
    name: jenkins
    groups: docker
    append: yes
```

7.4 roles/kubectl/tasks/main.yml

ansible/roles/kubectl/tasks/main.yml

```
---
- name: Install kubectl
  get_url:
    url: https://dl.k8s.io/release/v1.29.0/bin/linux/amd64/kubectl
    dest: /usr/local/bin/kubectl
    mode: '0755'

- name: Install Helm
  shell: |
    curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
  args:
    creates: /usr/local/bin/helm

- name: Install AWS CLI v2
  shell: |
    curl -s "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o /tmp/awscliv2.zip
    unzip -q /tmp/awscliv2.zip -d /tmp
    /tmp/aws/install
  args:
    creates: /usr/local/bin/aws

- name: Install Terraform
```

```
  unarchive:
    src: https://releases.hashicorp.com/terraform/1.7.0/terraform_1.7.0_linux_amd64.zip
    dest: /usr/local/bin/
    remote_src: yes
    mode: '0755'
```

8. Monitoring - Prometheus & Grafana

Prometheus scrapes metrics from all Node.js pods (via annotations) and from the EKS nodes (via kube-state-metrics and node-exporter). Grafana visualises the data with pre-built dashboards. Both are deployed with Helm into the monitoring namespace.

8.1 Install Commands

Install monitoring stack

```
# Add Helm repos
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana             https://grafana.github.io/helm-charts
helm repo update

# Install Prometheus stack (includes Alertmanager, kube-state-metrics, node-exporter)
helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace \
  -f helm/monitoring/prometheus-values.yaml

# Install Grafana (if not using bundled)
helm upgrade --install grafana grafana/grafana \
  --namespace monitoring \
  -f helm/monitoring/grafana-values.yaml
```

8.2 prometheus-values.yaml

helm/monitoring/prometheus-values.yaml

```
alertmanager:
  enabled: true
  alertmanagerSpec:
    replicas: 1

prometheus:
  prometheusSpec:
    replicas: 1
    retention: 15d
    storageSpec:
      volumeClaimTemplate:
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 20Gi
    additionalScrapeConfigs:
      - job_name: 'streamingapp-services'
        kubernetes_sd_configs:
          - role: pod
            namespaces:
              names: [streamingapp]
        relabel_configs:
          - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
```

```
            action: keep
            regex: true
          - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
            action: replace
            target_label: __metrics_path__
          - source_labels: [__address__, __meta_kubernetes_pod_annotation_prometheus_io_port]
            action: replace
            regex: ([^:]+)(?::\d+)?;(\d+)
            replacement: $1:$2
            target_label: __address__

grafana:
  enabled: true
  adminPassword: "CHANGE_ME_IN_PROD"
  persistence:
    enabled: true
    size: 5Gi
  dashboardProviders:
    dashboardproviders.yaml:
      apiVersion: 1
      providers:
        - name: default
          orgId: 1
          type: file
          options:
            path: /var/lib/grafana/dashboards
```

8.3 Prometheus Alert Rules

k8s/monitoring/alert-rules.yaml

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: streamingapp-alerts
  namespace: monitoring
  labels:
    release: kube-prometheus-stack
spec:
  groups:
    - name: streamingapp.rules
      rules:
        - alert: ServiceDown
          expr: up{job="streamingapp-services"} == 0
          for: 1m
          labels:
            severity: critical
          annotations:
            summary: "Service {{ $labels.kubernetes_pod_name }} is down"

        - alert: HighCPUUsage
          expr: |
            rate(container_cpu_usage_seconds_total{namespace="streamingapp"}[5m]) > 0.8
          for: 5m
          labels:
```

```
        severity: warning
      annotations:
        summary: "High CPU on {{ $labels.pod }}: {{ $value | humanizePercentage }}"

    - alert: HighMemoryUsage
      expr: |
        container_memory_usage_bytes{namespace="streamingapp"}
        / container_spec_memory_limit_bytes{namespace="streamingapp"} > 0.85
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Memory pressure on {{ $labels.pod }}"

    - alert: PodCrashLooping
      expr: rate(kube_pod_container_status_restarts_total{namespace="streamingapp"}[10m]) > 0
      for: 2m
      labels:
        severity: critical
      annotations:
        summary: "Pod {{ $labels.pod }} is crash-looping"
```

9. Sprint Plan - Detailed Task Breakdown

9.1 Sprint 1: Architecture, Dockerization & Jenkins Setup [Weeks 1-2]

* Draw AWS architecture diagram (EKS, ECR, ALB, VPC, S3, Secrets Manager)
* Review and improve all 5 Dockerfiles (add HEALTHCHECK, non-root user, .dockerignore)
* Create nginx.conf for frontend SPA routing and API proxying
* Create ECR repositories for all 5 services via AWS Console or Terraform bootstrap
* Launch EC2 t3.medium (Ubuntu 22.04) for Jenkins in public subnet
* Install Jenkins via Ansible (roles/jenkins) or user-data script
* Install Jenkins plugins: Git, Docker Pipeline, Kubernetes CLI, AWS Credentials, Slack
* Configure Jenkins global credentials: AWS, Docker, GitHub SSH key
* Add GitHub webhook -> Jenkins (push-triggered build)
* Validate: push a code change, confirm Jenkins picks it up and builds the Docker image

9.2 Sprint 2: AWS Infrastructure with Terraform [Weeks 3-4]

* Create S3 bucket for Terraform state + DynamoDB table for state locking
* Write terraform/vpc.tf - VPC, 2 public + 2 private subnets, Internet Gateway, NAT Gateway
* Write terraform/eks.tf - EKS 1.29 cluster, managed node group (t3.medium x2-6)
* Write terraform/ecr.tf - 5 ECR repos with lifecycle policy (keep last 10 images)
* Write terraform/s3.tf - S3 bucket (streamingapp-videos) with versioning, encryption
* Write terraform/iam.tf - IAM role for EKS node group + S3 access policy
* Write terraform/security.tf - Security groups for Jenkins EC2, EKS nodes
* Create Jenkins job 'infra-provision' that runs terraform plan then terraform apply
* Test: run infra-provision job, verify EKS cluster, VPC appear in AWS Console
* Store outputs (cluster endpoint, VPC ID) as Jenkins env vars for downstream jobs

9.3 Sprint 3: Ansible Configuration Management [Weeks 5-6]

* Configure ansible.cfg and AWS EC2 dynamic inventory (aws_ec2.yml)
* Write roles/common - OS updates, NTP, hostname, swap disable
* Write roles/docker - Install Docker CE 25+, add jenkins user to docker group
* Write roles/kubectl - Install kubectl 1.29, Helm 3, AWS CLI v2, Terraform 1.7
* Write roles/jenkins - Install Java 17, Jenkins LTS, systemd unit, initial admin
* Write site.yml targeting Jenkins EC2 (tag_Role_jenkins) and app nodes
* Create Jenkins job 'configure-nodes' that runs ansible-playbook site.yml
* Chain jobs: infra-provision -> configure-nodes (post-build trigger)
* Test: destroy and re-create EC2, run configure-nodes, verify all tools installed
* Document inventory tags and how to add new nodes to the fleet

9.4 Sprint 4: CI/CD Pipeline for EKS Deployment [Weeks 7-8]

* Write Jenkinsfile with parallel test stages for all services
* Add Docker build stage: build + push all 5 images to ECR with git-sha tag
* Add terraform plan/apply stage (gated by input approval for prod)
* Add ansible configure stage (runs after terraform apply)
* Write k8s/ YAML files: namespace, secret, mongo StatefulSet, 5 Deployments, Services
* Write ingress.yaml for AWS ALB Ingress Controller with HTTPS and path routing

* Add HPA for auth, streaming, chat services
* Add Helm deployment stage: helm upgrade --install with --wait
* Add smoke test stage: curl health checks on all services
* Test end-to-end: push to main -> all stages pass -> pods healthy in EKS

9.5 Sprint 5: Monitoring with Prometheus & Grafana [Weeks 9-10]

* Deploy kube-prometheus-stack via Helm into monitoring namespace
* Add Prometheus scrape annotations to all service Deployment YAMLs
* Add /metrics endpoint to Node.js services using prom-client library
* Create PrometheusRule YAML for ServiceDown, HighCPU, HighMemory, CrashLoop alerts
* Configure Alertmanager to send alerts to Slack (#devops-alerts channel)
* Import Grafana dashboards: Kubernetes cluster (ID 13770), Node Exporter (ID 1860)
* Create custom Grafana dashboard: StreamingApp - requests/sec, latency, error rate
* Integrate Jenkins with Grafana annotations: post deployment marker on charts
* Configure Jenkins post-build Slack notification on failure with build URL
* Test: simulate pod failure, verify alert fires in Grafana and Slack notification received

9.6 Sprint 6: Testing, Documentation & Final Automation [Weeks 11-12]

* Write integration tests: test auth flow, video listing, chat room creation end-to-end
* Write infrastructure test with Terratest: verify EKS cluster is reachable
* Add test reports to Jenkins (JUnit XML output)
* Document all Jenkins jobs, Terraform vars, and Helm values in /docs folder
* Create RUNBOOK.md: how to roll back a deployment, how to scale services
* Automate full pipeline: push to main -> test -> build -> infra -> configure -> deploy -> verify
* Conduct load test with k6 or Locust; verify HPA scales up pods automatically
* Review cost: right-size node group, confirm ECR lifecycle policy prunes old images
* Final demo: show end-to-end pipeline run in Jenkins with all stages green
* Submit documentation: architecture diagram, sprint reports, monitoring screenshots

10. Cost Optimisation (10% of Grade)

AWS costs must be actively managed. The following strategies reduce spend without compromising reliability for the capstone demo.

EKS Node Group	Use t3.medium Spot instances for non-prod; use On-Demand only for prod nodes. Set min=0 for dev clusters during off-hours with Cluster Autoscaler.
NAT Gateway	Use single_nat_gateway=true in Terraform for staging. Split per-AZ only in prod. Saves ~$32/month per AZ.
ECR Lifecycle Policy	Keep only last 10 images per repo. Prevents GB accumulation of unused layers.
S3 Storage Class	Use S3 Intelligent-Tiering for video assets. Automatically moves cold objects to cheaper tiers.
Prometheus Retention	Set retention=15d. Avoids unbounded disk growth on the EKS volume.
Jenkins EC2	Stop the Jenkins EC2 overnight with a Lambda/EventBridge schedule when not in use (saves ~50% EC2 cost).
EKS Cluster Idle	Scale node group to 0 in dev/staging environments after hours using Cluster Autoscaler min=0.
MongoDB	Use a single shared MongoDB pod for dev/staging. Use DocumentDB or Atlas only in prod.
ALB	Consolidate all services behind a single ALB using path-based routing (done via Ingress). Avoids multiple ALBs at $16/month each.
Cost Monitoring	Enable AWS Cost Explorer and set a billing alarm at $50/month for the capstone account.

11. Deliverables Summary

Jenkins CI/CD Pipeline	Jenkinsfile at repo root with 9 stages; triggered by GitHub webhook on push to main
Docker Images (5)	frontend, auth-service, streaming-service, admin-service, chat-service; pushed to AWS ECR
Kubernetes Manifests	k8s/ folder: namespace, secrets, mongo StatefulSet, 5 Deployments, 5 Services, Ingress, 3 HPAs
Helm Chart	helm/streamingapp/ with parameterised templates; values-dev.yaml, values-prod.yaml
Terraform Scripts	terraform/ folder: VPC, EKS, ECR, S3, IAM, Security Groups; remote state in S3
Ansible Playbooks	ansible/ folder: roles for common, docker, kubectl, jenkins; dynamic AWS inventory
Monitoring Stack	Prometheus + Grafana via Helm; custom dashboard; PrometheusRule alerts; Slack integration
Documentation	README, RUNBOOK.md, architecture diagram, per-sprint reports, troubleshooting guide
Test Reports	JUnit XML from npm test; Terratest infrastructure tests; k6 load test results
Cost Report	AWS Cost Explorer screenshots; Spot vs On-Demand analysis; lifecycle policy verification

Repository Structure

```
StreamingApp/
+-- Jenkinsfile
+-- docker-compose.yml        (local dev)
+-- frontend/
+-- backend/
|   +-- authService/
|   +-- streamingService/
|   +-- adminService/
|   +-- chatService/
+-- k8s/                      (Kubernetes YAML)
+-- helm/                     (Helm charts)
|   +-- streamingapp/
|   +-- monitoring/
+-- terraform/                (IaC)
+-- ansible/                  (Config management)
+-- docs/                     (Documentation)
```